Unit No. VI Database Connectivity

Database Connectivity refers to the process that allows applications (like software programs, websites, or mobile apps) to connect and interact with a **Database Management System (DBMS)** to **store, retrieve, update, and delete** data.

This connection is typically made using a **Database Driver** (like JDBC, ODBC, or a database-specific driver), and it allows programs to perform **queries and operations** on the database.

Unit No. VI Why is it Important?

Database connectivity is crucial because it:

- Enables dynamic data-driven applications.
- Allows users to access updated and real-time information.
- Supports scalability and centralized data management.

Common Interfaces/Technologies.

- **JDBC (Java Database Connectivity)** – Used in Java.
- **ODBC (Open Database Connectivity)** – Platform-independent.
- **ADO.NET** – Used in .NET applications.
- **Python libraries** like sqlite3, MySQL Connector, SQLAlchemy.

Features of Database Connectivity

Feature	Description
Real-time Access	Allows applications to access live data in real-time.
Query Execution	Supports SQL queries for operations like SELECT, INSERT, UPDATE, DELETE.
Security	Ensures data access is authenticated and authorized.
Data Integrity	Maintains accuracy and consistency of data.
Multi-user Access	Supports concurrent access by multiple users.
Cross-platform Support	Can connect applications from different platforms to the same database.
Transaction Support	Ensures multiple operations can be performed safely (commit or rollback).

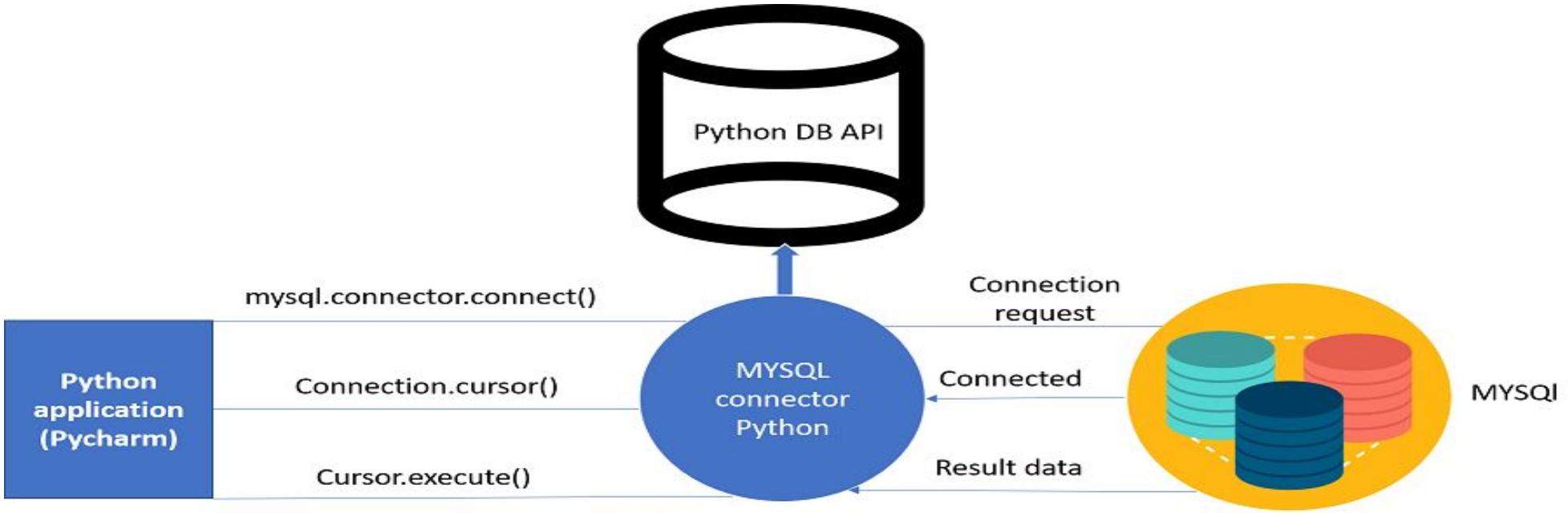
Features of Database Connectivity

Types of Database Drivers (based on language or platform):

1. Python Database Drivers

Database	Driver Example	Package name
MySQL	`mysql-connector-python`	`mysql-connector-python`
	`PyMySQL`	`pymysql`
PostgreSQL	`psycopg2`	`psycopg2`
SQLite	Built-in	`sqlite3` (built-in in Python)
Oracle	`cx_Oracle`	`cx_Oracle`
MongoDB	`pymongo`	`pymongo`

How does Python connect to a database?



Connectivity database connectivity in python with MySQL

1. Install MySQL Connector

You can install it using pip:

```bash
pip install mysql-connector-python
```

---

2. Import the Connector

```python
import mysql.connector
```

3. Establish a Connection

You need to connect to your MySQL database using credentials:

```python
conn = mysql.connector.connect(
    host="localhost",
    user="your_username",
    password="your_password",
    database="your_database"
)
```

4. Create a Cursor Object

The cursor is used to execute SQL queries:

```python
cursor = conn.cursor()
```

---

5. Execute SQL Queries

You can perform operations like SELECT, INSERT, UPDATE, DELETE, etc.

```python
cursor.execute("SELECT * FROM students")
result = cursor.fetchall()

for row in result:
    print(row)
```

6. Insert Data Example

```python
sql = "INSERT INTO students (name, age) VALUES (%s, %s)"
values = ("Alice", 22)
cursor.execute(sql, values)
conn.commit()
```

---

7. Close the Connection

Always close the cursor and connection when done:

```python
cursor.close()
conn.close()
```

PreparedStatement

- PreparedStatement is a **subinterface of Statement** in Java's JDBC API.
- It is used to **execute precompiled SQL queries** with or without parameters.
- Unlike Statement, a PreparedStatement can **accept input values at runtime** and helps prevent **SQL injection attacks**.

PreparedStatement

✅ **Key Advantages:**

1. **Security**: Prevents **SQL injection** attacks by safely escaping input.
2. **Reusability**: You can use the same query with different values.
3. **Readability**: Cleaner and easier to manage code.
4. **Efficiency**: The SQL query is compiled once and reused with different data.

ResultSet Metadata

ResultSet metadata refers to **information about the structure of the data** returned by a database query — not the data itself, but the *description* of that data.

This includes:

- Number of columns
- Column names
- Data types
- Table name
- Column sizes
- Whether a column is nullable, auto-incremented, etc.

This metadata is very useful when:

- You don't know the structure of the query result in advance.
- You want to write dynamic code that can handle any query.

ResultSet Metadata

Method	Description
`getColumnCount()`	Returns number of columns
`getColumnName(int column)`	Returns name of the column
`getColumnTypeName(int column)`	Returns SQL data type name
`getColumnClassName(int column)`	Java class name of column type
`isNullable(int column)`	Whether column can be null
`isAutoIncrement(int column)`	Whether the column is auto-incremented

ResultSet Metadata

In Python (using `cursor.description`)

```python
# Fetch metadata
for column in cursor.description:
    print(f"Name: {column[0]}, Type Code: {column[1]}")
```

Fields in `cursor.description` (Python)

Each column tuple contains:

1. **Name**
2. **Type Code**
3. **Display Size**
4. **Internal Size**
5. **Precision**
6. **Scale**
7. **Nullability**

Why ResultSet Metadata Matters:

- It enables **dynamic applications** (e.g., SQL query tools, report generators).
- You can build flexible UI forms or tables without hardcoding columns.
- Helps in **data validation**, logging, and debugging.

Database Metadata

Database Metadata refers to **information *about* the database itself**, not just the data it holds.

It describes:

- The **structure of the database**
- The **tables, columns, data types**
- The **views, indexes, stored procedures**
- The **users and their privileges**
- **Supported features** (SQL versions, functions, etc.)

In short, it's like a **data dictionary** or a "map" of the database — helping developers, DBAs, and applications understand how the database is designed and what it supports.

Metadata Type	Description
Tables	Names, schemas, types (BASE TABLE, VIEW)
Columns	Names, data types, sizes, nullability, default values
Primary/Foreign Keys	Key constraints on tables
Indexes	Information about indexing on columns
Views	Stored queries acting like virtual tables
Users/Roles	Who can access what
Stored Procedures/Functions	Their names, parameters, return types
Database Capabilities	Supported SQL syntax, functions, limits, etc.

```python
# Get columns from a specific table
cursor.execute("DESCRIBE students")
columns = cursor.fetchall()

for column in columns:
    print(column)
```

You can also query **INFORMATION_SCHEMA**:

```sql
SELECT * FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA = 'school';
SELECT * FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'students';
```

Feature	ResultSet Metadata	Database Metadata
Scope	Specific query result	Entire database
Used For	Describing columns in a query	Describing tables, schemas, DB capabilities
Java Interface	`ResultSetMetaData`	`DatabaseMetaData`
Python Way	`cursor.description`	Querying system tables (`INFORMATION_SCHEMA`)

Batch Processing

Batch processing is a technique where a **group of tasks (queries/operations)** are collected together and **executed as a single unit**, rather than running each one individually.

This is **more efficient**, especially for large volumes of repetitive tasks like inserting, updating, or deleting multiple rows in a database.

Why Use Batch Processing?

- **Improves performance**: Reduces the number of round-trips between your application and the database.
- **Reduces overhead**: Less network traffic, fewer I/O operations.
- **Atomicity**: Optionally, all statements in the batch can be committed or rolled back together.

Why Use Batch Processing?

Batch processing refers to **executing a group (batch) of operations together** instead of processing them one at a time. It's commonly used in:

- **Database operations** (like inserting multiple records)
- **ETL (Extract, Transform, Load)** jobs
- **Data warehousing**
- **Backend automation tasks**

Batch processing **improves performance** by reducing the number of database hits or I/O operations.

How Batch Processing Works

- You collect multiple SQL statements.
- Add them to a **batch** (like a queue).
- Send the batch to the database in one go.
- Execute all operations together.
- Optionally, commit or rollback based on success/failure.

How Batch Processing Works

```
# Execute all in one batch
cursor.executemany(query, data)
conn.commit()
```

END Chapter